# Mastering AWS DevOps: Automation & Scripting with Python

Sriharimalapati

Unlock the power of AWS DevOps with Python scripting! In this we'll explore how to automate cloud infrastructure, streamline CI/CD pipelines, and optimize deployments using Python. From AWS SDK (Boto3) to scripting EC2, S3, Lambda, and IAM automation, this article covers real-world examples and best practices for DevOps engineers. Whether you're automating deployments, monitoring resources, or enhancing security, this guide has you covered!

💡 **Key Takeaways:**

✅ Automate AWS infrastructure with Python (Boto3)
✅ Integrate Python into CI/CD pipelines
✅ Optimize AWS services with automation scripts
✅ Best practices for DevOps scripting


Let’s dive into AWS DevOps automation!

**What is AWS DevOps, and how does Python fit into it?**

AWS DevOps is a set of practices that automate and streamline software development, deployment, and infrastructure management on AWS. Python plays a crucial role by enabling automation through scripts using **Boto3** (AWS SDK for Python), infrastructure-as-code (IaC) tools, and integration with CI/CD pipelines.

**Q1: How can Python be used to automate EC2 instance management?**

**A:** Python can automate EC2 operations such as instance creation, termination, and status checks.

Example: Launching an EC2 instance using Boto3

```
import boto3

ec2 = boto3.resource('ec2')
instance = ec2.create_instances(
    ImageId='ami-0abcdef1234567890',
    MinCount=1,
    MaxCount=1,
    InstanceType='t2.micro',
    KeyName='my-key'
)
print("EC2 Instance Created:", instance[0].id)
```

## Q2: How do you use Boto3 to list all EC2 instances?

**A:** Boto3 is the AWS SDK for Python. You can list all EC2 instances using:

```
import boto3

ec2 = boto3.client('ec2')

response = ec2.describe_instances()
for reservation in response['Reservations']:
    for instance in reservation['Instances']:
        print(f"Instance ID: {instance['InstanceId']}, State: {instance['State']['Name']}")
```

## Q3: How do you automate S3 bucket creation using Python?

```
import boto3

s3 = boto3.client('s3')
bucket_name = "my-unique-bucket-12345"

s3.create_bucket(Bucket=bucket_name)
print(f"Bucket {bucket_name} created successfully!")
```

## Q4: How do you trigger a Lambda function using Python?

```
import boto3

lambda_client = boto3.client('lambda')
response = lambda_client.invoke(
```

```
    FunctionName='myLambdaFunction',
    InvocationType='Event'
)

print("Lambda triggered:", response)
```

## Q5: How do you use Python to stop EC2 instances based on a tag?

```
import boto3

ec2 = boto3.client('ec2')

instances = ec2.describe_instances(Filters=[{'Name': 'tag:Environment', 'Values': ['dev']}])

for reservation in instances['Reservations']:
    for instance in reservation['Instances']:
        ec2.stop_instances(InstanceIds=[instance['InstanceId']])
        print(f"Stopping instance: {instance['InstanceId']}")

##### This stops all EC2 instances with the tag Environment=dev.
```

## Q6: How do you automate IAM user creation with Python?

```
import boto3

iam = boto3.client('iam')

response = iam.create_user(UserName='newuser')
print(f"User created: {response['User']['UserName']}")

#### This creates a new IAM user.
```

## Q7: How do you automate Route 53 DNS record creation using Python?

```
import boto3

route53 = boto3.client('route53')

response = route53.change_resource_record_sets(
    HostedZoneId='ZXXXXXXXXXXXX',
    ChangeBatch={
        'Changes': [{
            'Action': 'CREATE',
            'ResourceRecordSet': {
                'Name': 'example.mydomain.com',
```

```
            'Type': 'A',
            'TTL': 300,
            'ResourceRecords': [{'Value': '192.168.1.1'}]
        }
    }]
  }
)

print("DNS record created:", response)

### This creates an A record in Route 53.
```

## Q8: How do you use Python to fetch AWS CloudWatch logs?

```
import boto3

logs = boto3.client('logs')

response = logs.describe_log_groups()
for log_group in response['logGroups']:
  print(f"Log Group: {log_group['logGroupName']}")

## This fetches all available CloudWatch log groups.
```

## Q9: How do you automate AMI creation using Python?

```
import boto3

ec2 = boto3.client('ec2')
instance_id = "i-0abcd1234efgh5678"

response = ec2.create_image(
  InstanceId=instance_id,
  Name="MyBackupAMI",
  NoReboot=True
)

print("AMI ID:", response['ImageId'])

## This creates an Amazon Machine Image (AMI) from an EC2 instance.
```

## Q10: How do you upload a file to an S3 bucket using Python?

```
import boto3

s3 = boto3.client('s3')
```

```
s3.upload_file("local_file.txt", "my-bucket", "uploaded_file.txt")
print("File uploaded successfully!")

## This uploads local_file.txt to my-bucket.
```

## Q11: How do you automate AWS Lambda deployment using Python?

```
import boto3

lambda_client = boto3.client('lambda')

with open('lambda_function.zip', 'rb') as f:
    zip_data = f.read()

response = lambda_client.update_function_code(
    FunctionName='myLambdaFunction',
    ZipFile=zip_data
)

print("Lambda function updated:", response)

### This updates an AWS Lambda function with new code.
```

## Q12: How do you monitor AWS services using Python and CloudWatch?

```
import boto3

cloudwatch = boto3.client('cloudwatch')
response = cloudwatch.get_metric_statistics(
    Namespace='AWS/EC2',
    MetricName='CPUUtilization',
    Dimensions=[{'Name': 'InstanceId', 'Value': 'i-1234567890abcdef0'}],
    StartTime='2024-02-01T00:00:00Z',
    EndTime='2024-02-02T00:00:00Z',
    Period=3600,
    Statistics=['Average']
)
print("CPU Utilization Data:", response)
```

## Q13: How do you integrate Python with CI/CD pipelines for AWS?

Python can be used in Jenkins, GitHub Actions, and AWS CodePipeline for tasks like:

- 🔹 Running automated tests with PyTest
- 🔹 Deploying infrastructure using Terraform with Python scripts
- 🔹 Triggering AWS Lambda functions for post-deployment automation

Example: A Python script to trigger an AWS Lambda function during deployment

```
import boto3

lambda_client = boto3.client('lambda')
response = lambda_client.invoke(
    FunctionName='myLambdaFunction',
    InvocationType='RequestResponse'
)
print("Lambda Function Triggered:", response)
```

## Conclusion: Mastering AWS DevOps Automation with Python

Automating AWS with Python scripting is a game-changer for DevOps engineers, enabling efficient infrastructure management, CI/CD optimization, and security enhancements. With powerful tools like **Boto3**, AWS Lambda, and CloudWatch, Python simplifies tasks such as **EC2 management, S3 automation, IAM security, and cost optimization**.

By integrating Python into your DevOps workflows, you can:
✅ Reduce manual intervention and human errors
✅ Improve deployment speed and system reliability
✅ Optimize AWS resources for cost efficiency
✅ Enhance security through automated IAM policies

……
……

You're welcome! Have a great time ahead! Enjoy your day!

Please Connect with me any doubts.

Mail: sriharimalapati6@gmail.com

LinkedIn: www.linkedin.com/in/

GitHub: https://github.com/Consultantsrihari

Medium: Sriharimalapati — Medium

## Thanks for watching ##### %%%% Sri Hari %%%%

AWS

Python

Automation

Scripting

DevOps

## Written by Sriharimalapati